

Producer-Consumer 유한 버퍼 문제 설명

0. Intro

생산자-소비자 문제는 멀티스레드 프로그래밍에서 중요한 동기화 문제입니다. 이 문제를 이해하기 위해서는 먼저 스레드와 멀티스레딩이 왜 등장했는지, 그리고 이들이 본질적으로 어떤 문제를 야기하는지 파악해야 합니다. 프로세스의 한계로부터 스레드 개념이 탄생한 역사적 배경을 살펴보고자 합니다. 멀티코어 시대의 도래와 함께 멀티스레딩이 필수가 된 과정을 조명합니다. 나아가 공유 메모리, 임계 구역, 교착상태 등 멀티스레딩의 근본적 문제들을 깊이 있게 분석합니다. 이러한 토대 위에서 생산자-소비자 문제를 다룰 것입니다. 마지막으로 바쁜 대기, 세마포, 모니터에 이르는 해결 전략의 진화를 체계적으로 고찰합니다.

1. 프로세스의 한계와 스레드의 등장

1.1 멀티프로그래밍의 시작

초기 컴퓨터 시스템은 배치 처리 방식을 사용했습니다. 하나의 작업이 완전히 끝날 때까지 다른 작업은 대기해야 했습니다. IBM의 연구자들은 중요한 사실을 발견했습니다. 대부분의 프로그램은 실행 시간의 상당 부분을 입출력 대기에 소비한다는 것이었습니다. 당시 자기 테이프나 카드 리더의 속도는 CPU에 비해 수천 배 느렸습니다. 프로그램이 디스크에서 데이터를 읽거나 프린터로 출력하는 동안 고가의 CPU는 아무 일도 하지 않고 유향 상태로 있어야 했습니다. 이는 심각한 자원 낭비였습니다.

이 문제를 해결하기 위해 멀티프로그래밍 개념이 도입되었습니다. 여러 프로그램을 메모리에 동시에 적재하고 한 프로그램이 입출력을 기다리는 동안 다른 프로그램에 CPU를 할당하는 것입니다. 운영체제는 각 프로그램을 독립된 실행 단위인 프로세스로 추상화했습니다. 각 프로세스는 자신만의 독립된 주소 공간을 가졌습니다. 코드, 데이터, 힙, 스택 영역이 모두 분리되었습니다. 이는 철저한 격리를 제공했습니다. 한 프로세스에서 발생한 오류가 다른 프로세스에 영향을 미치지 않게 된 것입니다. 덕분에 시스템 전체의 안정성이 크게 향상되었습니다.

1.2 프로세스의 무거움

그러나 멀티프로세싱에는 두 가지 명백한 한계가 있었습니다. 첫째, 프로세스 생성과 전환 비용이 매우 높았습니다. 새로운 프로세스를 만들려면 독립된 주소 공간 전체를 할당해야 했

습니다. 페이지 테이블, 파일 디스크립터 테이블, 시그널 핸들러 등 커널 자료구조들을 복사해야 했습니다. 이는 수천에서 수만 개의 명령어를 실행하는 작업이었습니다. 문맥 교환도 무거웠습니다. 한 프로세스에서 다른 프로세스로 전환할 때 모든 레지스터 상태를 저장하고 새 프로세스의 상태를 복원해야 했습니다. 더 심각한 것은 주소 공간이 완전히 바뀌므로 CPU 캐시와 TLB가 모두 무효화된다는 점이었습니다. 즉, 새 프로세스가 실행을 시작하면 캐시 미스가 빈번하게 발생하여 성능이 급격히 저하되었습니다.

둘째, 프로세스 간 통신이 복잡하고 비효율적이었습니다. 독립된 주소 공간은 안정성을 제공했습니다. 하지만 데이터 공유를 극도로 어렵게 만들었습니다. 두 프로세스가 정보를 교환하려면 별도의 IPC 메커니즘이 필요했습니다. 파이프를 사용하면 데이터를 커널 버퍼를 거쳐 전달해야 했습니다. 생산자 프로세스가 데이터를 사용자 공간에서 커널 공간으로 복사하고, 소비자 프로세스가 다시 커널 공간에서 사용자 공간으로 복사해야 했습니다. 두 번의 복사 오버헤드가 발생했습니다. 공유 메모리를 사용하면 복사는 피할 수 있었지만 설정이 복잡했고 동기화 문제가 여전히 남았습니다. 소켓은 네트워크 통신에 적합했지만 지역적 통신에는 과도한 오버헤드였습니다.

1.3 스레드의 탄생

철저한 격리는 항상 필요하지 않았습니다. 같은 프로그램 내의 여러 실행 흐름은 코드와 데이터를 안전하게 공유할 수 있었습니다. 독립된 주소 공간을 새로 만드는 것은 불필요한 낭비였습니다. 이러한 인식에서 스레드 개념이 탄생했습니다.

스레드는 프로세스 내에서 실행되는 경량 실행 단위였습니다. 한 프로세스에 속한 스레드들은 코드, 데이터, 힙 영역을 공유했습니다. 반면 각 스레드는 독립적인 실행 흐름을 위해 자신만의 스택, 레지스터, 프로그램 카운터를 가졌습니다. 이러한 설계는 극적으로 효율성을 향상시켰습니다. 스레드 생성은 프로세스 생성보다 10배에서 100배 빠랐습니다. 새로운 주소 공간을 할당할 필요가 없었기 때문입니다. 스택과 스레드 제어 블록만 만들면 되었습니다. 스레드 간 문맥 교환도 훨씬 가벼웠습니다. 주소 공간이 바뀌지 않으므로 페이지 테이블을 교체할 필요가 없었습니다. 캐시와 TLB가 대부분 유효한 상태로 남았습니다. 데이터 공유는 자연스러워졌습니다. 전역 변수나 힙 객체를 통해 직접 통신할 수 있었습니다. IPC의 복잡성과 오버헤드를 줄일 수 있게 된 것입니다.

2. 멀티코어 시대와 멀티스레딩의 필연성

2.1 클럭 속도의 한계

수십 년간 계속되던 클럭 속도 향상이 물리적 한계에 부딪혔습니다. 1980년대 초 수 MHz였던 클럭 속도는 2000년대 초 수 GHz까지 상승했습니다. 그러나 속도 향상은 전력 소비와 발열 문제를 일으켰습니다. 전력 소비는 클럭 속도의 세제곱에 비례했습니다. 속도를 두 배로 높이면 전력은 여덟 배가 되었습니다. 발열을 감당할 수 없게 되었습니다. 무어의 법칙에 따라 트랜지스터 수는 계속 증가했지만, 클럭 속도는 정체되었습니다.

해결책은 명확했습니다. 하나의 빠른 코어 대신 여러 개의 느린 코어를 사용하는 것이었습니다. 이는 근본적인 패러다임 전환이었습니다. 과거에는 프로그램을 그대로 두어도 새 CPU에서 자동으로 빨라졌습니다. 클럭 속도가 높아졌기 때문입니다. 그러나 멀티코어 시대에는 그렇지 않았습니다. 단일 스레드 프로그램은 하나의 코어만 사용했습니다. 나머지 코어들은 유향 상태로 남았습니다. 성능 향상을 얻으려면 유향 상태의 코드를 사용하여 프로그램을 병렬화해야 했습니다.

2.2 멀티스레딩의 필요성

이는 소프트웨어 개발에 엄청난 도전을 제시했습니다. 순차적 프로그래밍은 직관적이고 단순했습니다. 명령어들이 순서대로 실행되었습니다. 결과는 예측 가능했습니다. 병렬 프로그래밍은 복잡하고 어려웠습니다. 여러 실행 흐름이 동시에 진행되었기 때문입니다. 타이밍에 따라 결과가 달라질 수 있었습니다. 그러나 선택의 여지가 없었습니다. 하드웨어가 변했고 소프트웨어도 따라야 했기 때문입니다. 멀티스레딩은 선택이 아닌 필수가 되었습니다.

멀티스레딩의 이점은 명확했습니다. 첫째, 성능 향상이었습니다. 계산 집약적인 작업을 여러 스레드로 분할하면 여러 코어가 동시에 작업했습니다. 이미지 처리, 과학 계산, 비디오 인코딩 같은 작업에서 극적인 속도 향상이 가능했습니다. 둘째, 응답성 향상이었습니다. GUI 애플리케이션에서 사용자 입력을 처리하는 메인 스레드와 백그라운드 작업을 수행하는 워커 스레드를 분리할 수 있었습니다. 사용자가 버튼을 클릭하면 즉시 반응했습니다. 실제 작업은 다른 스레드에서 진행되었습니다. 애플리케이션이 멈추지 않았습니다. 셋째, 자원 활용 극대화였습니다. 웹 서버는 수천 개의 동시 연결을 처리해야 했습니다. 각 연결마다 프로세스를 만들면 자원이 고갈되었습니다. 스레드 풀을 사용하면 적은 수의 스레드로 많은 요청을 처리할 수 있었습니다.

그러나 멀티스레딩은 근본적인 문제들을 야기했습니다. 스레드들이 메모리를 공유한다는 바로 그 특성이 복잡성의 근원이었습니다. 공유 없이는 효율적인 협력이 불가능했습니다. 하지만 공유는 함께 혼란을 불러왔습니다.

3. 멀티스레딩의 근본적 문제들

3.1 공유 메모리와 경쟁 상태

스레드의 장점은 동시에 문제를 만들었습니다. 메모리 공유는 효율적인 통신을 가능하게 했습니다. 동시에 제어되지 않은 동시 접근은 재앙을 불렀습니다. 가장 단순한 예를 생각해봅시다. 두 스레드가 전역 변수 count를 증가시킵니다. 각 스레드는 count++를 1000번 실행합니다. 직관적으로 최종 count 값은 2000이어야 합니다. 그러나 실제로는 1000에서 2000 사이의 임의의 값이 나타납니다. 실행할 때마다 결과가 다르게 나타납니다. 즉 예측을 할 수 없게 된 것입니다.

이유는 `count++` 연산이 원자적이지 않기 때문입니다. 고수준 언어에서는 하나의 문장입니다. 하지만 `mips`와 같은 수준에서는 세 개의 명령어로 분해됩니다. 첫째, 메모리에서 `count` 값을 레지스터로 읽습니다. 둘째, 레지스터 값을 1 증가시킵니다. 셋째, 레지스터 값을 메모리에 다시 씁니다. 두 스레드가 이 과정을 동시에 실행하면 인터리빙이 발생합니다. 스레드 A가 `count`를 읽어 레지스터 R1에 5를 저장합니다. 스레드 B도 `count`를 읽어 레지스터 R2에 5를 저장합니다. 스레드 A가 R1을 6으로 증가시킵니다. 스레드 B도 R2를 6으로 증가시킵니다. 스레드 A가 `count`에 6을 씁니다. 스레드 B도 `count`에 6을 씁니다. 두 번의 증가가 있었지만 `count`는 6입니다. 결과적으로 하나의 업데이트가 사라졌습니다.

이를 경쟁 상태라고 부릅니다. 결과가 스레드들의 실행 순서에 의존합니다. 순서는 스케줄러가 결정하며 예측할 수 없습니다. 같은 입력으로 실행해도 다른 출력이 나옵니다. 프로그램이 비결정적이 됩니다. 더 심각한 것은 재현하기 어렵다는 점입니다. 버그가 가끔만 나타납니다. 디버거를 붙이면 타이밍이 바뀌어 버그가 사라집니다. 이를 하이젠버그라 부릅니다. 관찰하려는 순간 상태가 변하는 것입니다.

경쟁 상태는 모든 공유 자료구조에서 발생할 수 있습니다. 연결 리스트에 동시에 삽입하면 포인터가 꼬입니다. 노드가 사라지거나 루프가 생깁니다. 해시 테이블을 동시에 수정하면 내부 불변식이 깨집니다. 크기 정보와 실제 원소 개수가 맞지 않습니다. 파일에 동시에 쓰면 데이터가 섞입니다. 로그 메시지가 중간에 끊깁니다. 공유 메모리가 있는 한 경쟁 상태의 위험은 항상 존재합니다.

3.2 임계 구역과 상호 배제

경쟁 상태를 방지하려면 공유 자원에 접근하는 코드 부분을 보호해야 합니다. 이를 임계 구역이라 부릅니다. 임계 구역은 한 번에 하나의 스레드만 실행할 수 있어야 합니다. 이를 상호 배제라 합니다. `count`를 증가시키는 세 명령어 전체가 하나의 임계 구역입니다. 한 스레드가 이 구역에 들어가면 다른 스레드는 대기해야 합니다.

Gary Peterson은 1981년 정교한 알고리즘을 제안했습니다. 두 개의 플래그와 하나의 턴 변수를 사용했습니다. 각 스레드는 자신의 플래그를 `true`로 설정하고 턴을 상대방에게 양보합니다. 상대방의 플래그가 `true`이고 턴이 상대방 것인 동안 대기합니다. 이론적으로는 완벽했습니다. 상호 배제, 진행 보장, 유한 대기를 모두 만족했습니다. 그러나 현대 하드웨어에서는 작동하지 않습니다. 컴파일러가 명령어 순서를 재배치하기 때문입니다. CPU도 성능을 위해 메모리 연산을 재정렬합니다. 플래그 설정과 턴 양보의 순서가 바뀌면 알고리즘이 무너집니다.

결국 소프트웨어만으로는 한계가 있었습니다. 하드웨어의 도움이 필요했습니다. 원자적 명령어가 도입되었습니다. `Test-And-Set`은 변수를 읽고 동시에 1로 설정하는 명령어입니다. 전체 과정이 분할 불가능합니다. 다른 코어가 개입할 수 없습니다. `Compare-And-Swap`은 더 강력합니다. 변수가 예상 값과 같으면 새 값으로 바꿉니다. 다르면 아무것도 하지 않습니다. 이런 하드웨어 지원 위에서 뮤텍스와 세마포가 등장하게 되었습니다.

3.3 교착상태

교착상태는 두 개 이상의 스레드가 서로가 가진 자원을 기다리며 영원히 진행하지 못하는 상황입니다.

교착상태는 네 가지 조건이 모두 만족될 때 발생합니다. 첫째, 상호 배제입니다. 자원은 한 번에 하나의 스레드만 사용할 수 있습니다. 둘째, 점유와 대기입니다. 스레드가 최소한 하나의 자원을 점유한 채로 다른 자원을 기다립니다. 셋째, 비선점입니다. 자원을 강제로 빼앗을 수 없습니다. 스레드가 자발적으로 해제할 때까지 기다려야 합니다. 넷째, 순환 대기입니다. 스레드들이 원형으로 서로의 자원을 기다립니다. 이 네 조건 중 하나라도 제거하면 교착상태를 예방할 수 있습니다.

가장 실용적인 방법은 순환 대기를 제거하는 것입니다. 모든 락에 순서를 부여합니다. 항상 낮은 번호의 락을 먼저 획득하도록 강제합니다. 락 L1이 락 L2보다 낮은 번호라면, L1을 먼저 획득해야 합니다. 이렇게 하면 순환이 형성될 수 없습니다. 그러나 이는 전역적인 순서를 정의하고 모든 코드가 따르도록 해야 합니다. 대규모 시스템에서는 구현하기 어렵습니다. 또 다른 방법은 타임아웃입니다. 일정 시간 내에 락을 획득하지 못하면 포기하고 다시 시도합니다. 이미 획득한 락들을 모두 해제합니다. 이는 교착상태를 깨지만 라이브락을 유발할 수 있습니다. 스레드들이 계속 충돌하고 재시도하며 진행하지 못하는 상황입니다.

교착상태는 재현하기 어렵기 때문에 디버깅이 매우 어렵습니다.

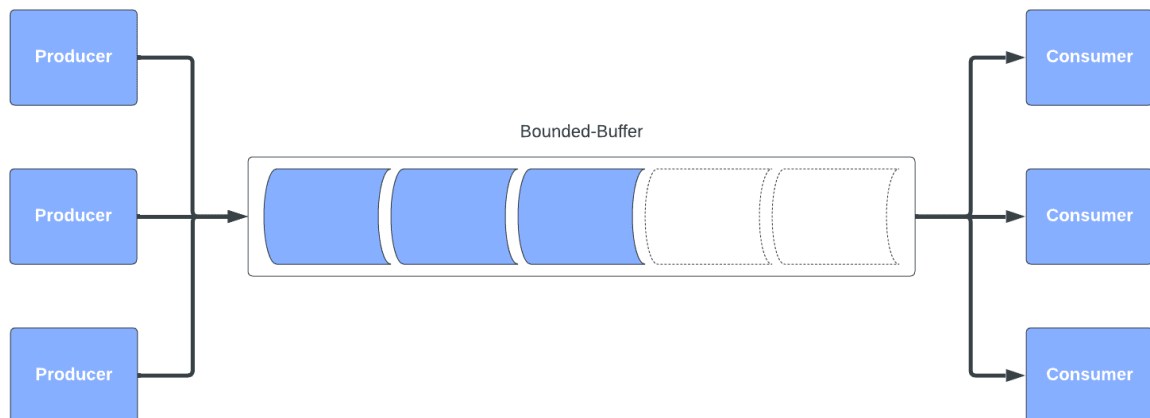
3.4 조건 동기화

상호 배제는 데이터 무결성을 보장합니다. 그러나 많은 경우 단순히 접근을 막는 것만으로는 부족합니다. 특정 조건이 만족될 때까지 기다려야 하는 상황이 있습니다. 이를 조건 동기화라 합니다. 버퍼가 가득 차면 생산자는 공간이 생길 때까지 기다려야 합니다. 버퍼가 비었으면 소비자는 항목이 들어올 때까지 기다려야 합니다.

가장 단순한 방법은 바쁜 대기(Polling)입니다. 조건을 반복적으로 확인하는 것입니다. 락을 획득하고 조건을 검사합니다. 만족되지 않으면 락을 해제하고 다시 획득하여 검사합니다. 만족될 때까지 반복합니다. 이는 극도로 비효율적입니다. 스레드가 실제 작업을 하지 않으면서 CPU 시간을 소비합니다. 단일 코어 시스템에서는 치명적입니다. 대기 중인 스레드가 CPU를 독점하면 조건을 만족시킬 스레드가 실행될 기회를 얻지 못합니다. 영원히 기다립니다.

효율적인 대기를 위해서 스레드를 재우는 방법을 사용합니다. 운영체제가 스레드를 블로킹하여 CPU를 양보하게 합니다. 조건이 만족되면 다른 스레드가 깨웁니다. 이를 위해 조건 변수가 사용됩니다. 조건 변수는 스레드가 특정 조건이 만족될 때까지 큐에서 대기하도록 돕는 동기화 장치입니다. 조건이 충족되지 않으면 스레드는 wait 연산을 호출하여 스스로 대기 상태로 전환하고 가지고 있던 락(Lock)을 원자적으로(atomicallly) 해제합니다. 락이 해제되었으므로 다른 스레드가 진입하여 상태를 변경할 수 있게 됩니다. 이후 조건을 충족시킨 다른 스레드가 signal을 보내면, 잠들어 있던 스레드가 깨어나 다시 락을 획득하고 작업을 재개합니다. 이를 통해 불필요한 CPU 소모 없이 효율적인 동기화가 가능해집니다.

4. 생산자-소비자 문제: 모든 것의 결합



<https://www.baeldung.com/cs/bounded-buffer-problem>

이제 생산자-소비자 문제를 다시 보면, 이것이 왜 멀티스레딩의 대표적 문제인지 명확해집니다. 이 문제는 지금까지 논의한 모든 근본적 문제들을 포함하고 있습니다.

4.1 문제의 본질

첫째, 공유 메모리 문제입니다. 생산자와 소비자가 버퍼를 공유합니다. 제어되지 않은 접근은 데이터 손상을 초래합니다. 둘째, 임계 구역 문제입니다. 버퍼를 조작하는 코드는 상호 배제가 필요합니다. 한 번에 하나의 스레드만 버퍼의 내부 상태를 변경해야 합니다. 셋째, 조건 동기화 문제입니다. 버퍼가 가득 차거나 비었을 때 효율적으로 대기해야 합니다. 넷째, 잠재적 교착상태 문제입니다. 잘못 설계하면 생산자와 소비자가 서로를 기다리며 멈출 수 있습니다.

4.2 문제의 정형화

에츠허르 데이크스트라는 1965년 이 문제를 명확히 정형화했습니다. 시스템은 세 가지 구성 요소로 이루어집니다. 생산자(Producer)는 데이터 항목을 생성하여 버퍼에 넣는 스레드입니다. 소비자(Consumer)는 버퍼에서 데이터 항목을 가져와 처리하는 스레드입니다. 유한 버퍼는 크기가 N 인 공유 버퍼입니다. 제약 조건은 명확합니다. 버퍼가 가득 차면 생산자는 빈 공간이 생길 때까지 대기해야 합니다. 버퍼가 비어있으면 소비자는 항목이 들어올 때까지 대기해야 합니다. 여러 생산자와 소비자가 동시에 버퍼에 접근할 수 있어야 합니다. 동시에, 데이터 무결성도 보장되어야 합니다.

4.3 현실의 생산자-소비자

생산자-소비자 패턴은 추상화된 문제가 아닙니다. 현대 시스템 곳곳에 존재합니다. 운영체제에서는 키보드 입력 처리에서 인터럽트 핸들러가 생산자 역할을, 애플리케이션이 소비자 역할을 합니다. 사용자가 키를 누르면 인터럽트가 발생합니다. 핸들러는 키 코드를 버퍼에 넣

습니다. 애플리케이션은 버퍼에서 읽어 처리합니다. 파일 시스템의 쓰기 캐시도 마찬가지입니다. 애플리케이션이 쓰기 요청을 버퍼에 넣으면, 플러시 데몬이 이를 실제 디스크에 씁니다. 네트워크 스택의 패킷 수신 버퍼도 같은 패턴입니다. 네트워크 카드가 패킷을 버퍼에 넣으면, 프로토콜 스택이 이를 처리합니다.

5. 원시적 해법: 바쁜 대기

초기 프로그래머들의 첫 번째 시도는 직관적이었습니다. 전역 변수로 버퍼 상태를 추적하고 조건이 만족될 때까지 반복문으로 확인하는 것이었습니다.

5.1 기본 구조

버퍼 배열과 함께 `count` 변수로 현재 항목 수를 추적합니다. `in`은 생산자가 쓸 위치이고, `out`은 소비자가 읽을 위치입니다. 생산자는 항목을 생성한 후 `while (count == N)` 루프로 버퍼가 가득 찬지 확인합니다. 생산자는 버퍼가 가득 차 있으면 아무것도 하지 않고 계속 확인합니다. 이것이 바쁜 대기입니다. 공간이 생기면 `buffer[in]`에 항목을 넣고, `in`을 순환 증가시키며, `count`를 증가시킵니다. 소비자는 대칭적으로 `while (count == 0)` 루프로 버퍼가 비었는지 확인합니다. 비어있으면 계속 확인합니다. 항목이 있으면 `buffer[out]`에서 가져와 `out`을 증가시키고 `count`를 감소시킵니다.

5.2 치명적 결함들

이 해법은 여러 치명적 결함이 있었습니다. 가장 심각한 것은 경쟁 상태입니다. `count` 증가와 감소는 원자적이지 않습니다. 앞서 논의했듯이 세 단계로 분해됩니다. 읽기, 수정, 쓰기입니다. 생산자와 소비자가 동시에 실행되면 둘 다 같은 `count` 값을 읽을 수 있습니다. 하나의 업데이트가 사라질 수 있습니다. `count` 값이 실제 버퍼 상태와 맞지 않게 됩니다. 버퍼가 실제로는 가득 찼는데 `count`가 `N-1`일 수 있습니다. 생산자가 또 쓰려고 하면 데이터가 덮어쓰워집니다. 버퍼가 비었는데 `count`가 1일 수 있습니다. 소비자가 읽으려 하면 잘못된 데이터를 읽습니다.

`in`과 `out`도 마찬가지입니다. 두 생산자가 동시에 `in`을 읽고 증가시키면 같은 위치에 쓸 수 있습니다. 하나의 항목이 사라집니다. 두 소비자가 동시에 `out`을 읽고 증가시키면 같은 항목을 두 번 읽을 수 있습니다. 데이터 무결성이 완전히 파괴됩니다. 버그는 간헐적으로 발생합니다. 타이밍에 따라 나타났다가 사라집니다. 재현하기 어렵기 때문에 디버깅이 어렵습니다.

또 다른 문제는 CPU 낭비입니다. 바쁜 대기는 스레드가 실제 작업을 하지 않으면서 CPU 시간을 소비합니다. `while` 루프가 끊임없이 실행됩니다. 조건을 확인하는 명령어들이 반복됩니다. 이는 다른 스레드가 사용할 수 있는 CPU 시간을 빼앗습니다. 단일 코어 시스템에서는 치명적입니다. 생산자가 바쁜 대기 중이면 소비자가 실행될 기회를 얻지 못할 수 있습니다. 소비자가 실행되어야 버퍼에 공간이 생기는데 생산자가 CPU를 독점합니다. 시스템이 진행

하지 못합니다. 멀티코어 시스템에서도 낭비입니다. 대기 중인 스레드가 하나의 코어를 완전히 소모합니다. 실제 작업을 하는 스레드가 사용할 수 있었을 자원을 낭비하게 됩니다.

6. 세마포를 이용한 고전적 해법

에츨허르 데이크스트라는 세마포를 발명했습니다. 이는 바쁜 대기의 문제들을 해결하는 첫 번째 실용적 도구였습니다.



6.1 세마포의 개념

세마포는 정수 카운터를 유지하고 두 가지 원자적 연산을 제공합니다. P 연산(wait)은 카운터를 감소시킵니다. 카운터가 0이면 스레드를 블로킹합니다. V 연산(signal)은 카운터를 증가시키고 대기 중인 스레드를 깨웁니다.

세마포는 구조체로 구현됩니다. 정수 count와 대기 중인 스레드들의 큐를 가집니다. P 연산의 동작은 다음과 같습니다. 먼저 count를 감소시킵니다. count가 음수가 되면 현재 스레드를 대기 큐에 추가합니다. block 함수를 호출하여 스레드를 재웁니다. 이때 스레드는 CPU를 양보합니다. 운영체제가 다른 스레드를 스케줄합니다. 대기 중인 스레드는 CPU 시간을 소비하지 않습니다. 이것이 바쁜 대기와의 핵심 차이입니다. V 연산은 count를 증가시킵니다. count가 0 이하면, 즉 대기 큐에 스레드가 있으면 큐에서 스레드 하나를 제거합니다. wakeup 함수를 호출하여 그 스레드를 준비 상태로 만듭니다. 스케줄러가 나중에 그 스레드를 실행합니다.

6.2 세 개의 세마포를 이용한 해법

데이크스트라는 세 개의 세마포로 생산자-소비자 문제를 해결했습니다. ❶ mutex는 초기 값 1로 상호 배제를 위해 사용됩니다. ❷ empty는 초기값 N으로 빈 슬롯 개수를 나타냅니다. ❸ full은 초기값 0으로 채워진 슬롯 개수를 나타냅니다. 버퍼 배열과 in(생산자가 넣을 위치), out(소비자가 꺼낼 위치) 변수가 사용됩니다.

생산자의 동작은 다음과 같습니다. 항목을 생성한 후 먼저 P(empty)를 호출합니다. 이는 빈 슬롯이 있을 때까지 대기하는 것입니다. empty의 값이 0이면, 즉 버퍼가 가득 차면 생산자는 블로킹됩니다. CPU를 양보하고 잠듭니다. 소비자가 항목을 제거하고 V(empty)를 호출하면 empty가 증가합니다. 대기 중인 생산자가 깨어납니다. 빈 슬롯을 확보한 생산자는 P(mutex)를 호출합니다. 이는 버퍼에 대한 배타적 접근을 획득하는 것입니다. 다른 스레드가 이미 mutex를 가지고 있으면 대기합니다. mutex를 획득하면 안전하게 버퍼를 조작할 수 있습니다. buffer[in]에 항목을 넣습니다. in을 $(in + 1) \% N$ 으로 순환 증가시킵니다. V(mutex)로 mutex를 해제합니다. 다른 스레드가 버퍼에 접근할 수 있게 됩니다. V(full)로 full을 증가시킵니다. 이는 대기 중인 소비자를 깨웁니다. 버퍼에 항목이 있음을 알립니다.

소비자는 대칭적으로 동작합니다. P(full)로 채워진 슬롯이 있을 때까지 대기합니다. full이 0이면, 즉 버퍼가 비었으면 블로킹됩니다. 생산자가 항목을 넣고 V(full)를 호출하면 깨어납니다. P(mutex)로 배타적 접근을 획득합니다. buffer[out]에서 항목을 가져옵니다. out을 증가시킵니다. V(mutex)로 해제합니다. V(empty)로 빈 슬롯을 하나 추가합니다. 대기 중인 생산자를 깨웁니다.

6.3 설계의 정교함

이 해법의 정교함은 세 개의 독립적 관심사를 분리했다는 점입니다. mutex는 버퍼 자료구조의 일관성을 보호합니다. 상호 배제를 담당합니다. 한 번에 하나의 스레드만 버퍼의 내부 상태를 변경할 수 있습니다. empty와 full은 자원 가용성을 추적합니다. 조건 동기화를 담당합니다. 버퍼에 공간이 있는지, 항목이 있는지를 나타냅니다. 각 세마포가 명확한 역할을 가집니다.

세마포에서는 호출 순서가 중요합니다. 생산자는 반드시 P(empty) 다음에 P(mutex)를 호출해야 합니다. 순서를 바꾸면 교착상태가 발생할 수 있습니다. 만약 P(mutex)를 먼저 호출한다고 가정해봅시다. 버퍼가 가득 찬 상황입니다. 생산자가 먼저 mutex를 획득합니다. 그 다음 P(empty)를 호출합니다. empty가 0이므로 블로킹됩니다. 그런데 mutex를 잡은 채로 블로킹됩니다. 소비자는 mutex를 획득할 수 없습니다. 항목을 소비할 수 없습니다. V(empty)를 호출할 수 없습니다. 생산자는 영원히 깨어나지 못합니다. 시스템이 멈춥니다. 즉 교착 상태가 발생합니다.

올바른 순서는 먼저 자원 가용성을 확인한 뒤, 배타적 접근을 획득하는 것입니다. P(empty)는 블로킹될 수 있지만 mutex를 잡고 있지 않습니다. 다른 스레드들이 자유롭게 동작할 수 있습니다. 소비자가 항목을 제거하고 V(empty)를 호출하면 생산자가 깨어납니다. 그 다음 P(mutex)를 호출하여 안전하게 버퍼를 조작합니다. 교착상태가 발생하지 않습니다.

7. 모니터와 조건 변수

C.A.R. Hoare와 Per Brinch Hansen은 모니터를 제안했습니다. 이는 세마포의 문제점들을 해결하는 구조화된 동기화 메커니즘이었습니다.

7.1 모니터의 개념

모니터는 공유 데이터와 접근 함수를 하나의 모듈로 캡슐화합니다. 모니터 내의 함수들은 상호 배제가 자동으로 보장됩니다. 한 번에 하나의 스레드만 모니터 내부에서 실행됩니다.

모니터 내부에는 버퍼 배열, `count`, `in`, `out` 같은 공유 변수가 있습니다. 이들은 모니터 외부에서 직접 접근할 수 없습니다. 완전히 캡슐화되어 있습니다. 조건 변수 `not_full`과 `not_empty`를 선언합니다. `not_full`은 버퍼가 가득 차지 않았다는 것을 나타냅니다. `not_empty`는 버퍼가 비지 않았다는 것을 나타냅니다.

`produce` 함수가 호출되면 모니터의 락이 자동으로 획득됩니다. 프로그래머가 명시적으로 락을 잡을 필요가 없습니다. `count`가 `N`, 즉 버퍼가 가득 차면 `wait(not_full)`을 호출합니다. `wait`는 세 가지 동작을 원자적으로 수행합니다. 현재 스레드를 `not_full`의 대기 큐에 넣습니다. 모니터의 락을 자동으로 해제합니다. 스레드를 블로킹합니다. 락 해제가 핵심입니다. 락을 잡은 채로 블로킹되면 다른 스레드가 모니터에 진입할 수 없습니다. 조건을 만족시킬 수 없습니다. 교착상태입니다. `wait`는 이를 방지합니다.

소비자가 항목을 제거하고 `signal(not_full)`을 호출하면 대기 중인 생산자 하나가 깨어납니다. 깨어난 스레드는 `wait`에서 반환되기 전에 모니터의 락을 다시 획득합니다. 이제 안전하게 조건을 재확인하고 작업을 진행할 수 있습니다. 버퍼에 공간이 있으면 항목을 넣습니다. 이후 `in`과 `count`를 증가시킵니다. `signal(not_empty)`로 대기 중인 소비자를 깨웁니다. 함수가 반환되면 모니터의 락이 자동으로 해제됩니다.

`consume` 함수도 비슷합니다. `count`가 0, 즉 버퍼가 비었으면 `wait(not_empty)`를 호출합니다. 락을 해제하고 블로킹됩니다. 생산자가 항목을 넣고 `signal(not_empty)`를 호출하면 대기 중인 소비자 하나가 깨어납니다. 깨어난 스레드는 `wait`에서 반환되기 전에 모니터의 락을 다시 획득합니다. 안전하게 조건을 확인하고 버퍼에서 항목을 제거합니다. 작업을 마친 소비자는 `signal(not_full)`을 호출하여 생산자가 새로운 데이터를 넣을 공간이 생겼음을 알립니다. 이 모든 과정이 끝나고 함수가 반환될 때 모니터의 락은 자동으로 해제되며 동기화 과정이 종료됩니다.

7.2 조건 변수의 시맨틱스

조건 변수에는 두 가지 시맨틱스가 있습니다. 1 Hoare 시맨틱스는 `signal`을 호출한 스레드가 즉시 블로킹됩니다. 깨어난 스레드가 즉시 실행됩니다. 조건이 확실히 만족된 상태가 보장됩니다. `signal`을 호출했다는 것은 조건을 막 만족시켰다는 의미입니다. 깨어난 스레드는 조건을 재확인할 필요가 없습니다. `if` 문으로 한 번만 검사하면 됩니다.

2 Mesa 시맨틱스는 signal을 호출한 스레드가 계속 실행됩니다. 깨어난 스레드는 나중에 스케줄됩니다. 따라서 조건이 다시 거짓이 될 수 있습니다. signal과 깨어난 스레드의 실행 사이에 시간 간격이 있습니다. 그 사이 다른 스레드가 개입할 수 있습니다. 이러한 단점에도 불구하고 대부분의 현대 시스템은 Mesa 시맨틱스를 사용합니다. 구현이 더 간단하고 문맥 교환이 줄어 효율적이기 때문입니다.

7.3 while 루프의 필요성

Mesa 시맨틱스는 중요한 프로그래밍 관행을 요구합니다. 반드시 while 루프로 조건을 재확인해야 합니다. Hoare와 달리 if 문으로 한 번만 검사하면 안 됩니다. consume 함수를 다시 봅시다. if (count == 0) wait(not_empty)로 작성하면 위험합니다. 소비자가 깨어났을 때 count가 여전히 0일 수 있습니다. 왜냐하면 생산자가 signal을 호출한 후 계속 실행되기 때문입니다. 그 사이 다른 소비자가 먼저 실행되어 항목을 가져갔을 수 있습니다. 첫 번째 소비자가 깨어나면 버퍼는 다시 비어 있습니다. count가 0입니다. 그런데 if 문이었다면 조건을 재확인하지 않습니다. buffer[out]을 읽으려 합니다. 잘못된 데이터를 읽거나 오류가 발생합니다.

while (count == 0) wait(not_empty)로 작성하면 안전합니다. 깨어난 후에도 조건을 재확인합니다. count가 0이면 다시 wait를 호출합니다. 다시 잠듭니다. count가 0이 아닐 때까지 반복합니다.

Spurious wakeup도 while 루프가 필요한 이유입니다. 시스템이 이유 없이 스레드를 깨울 수 있습니다. 조건이 만족되지 않았는데도 깨어날 수 있습니다. while 루프는 이런 경우에도 안전을 보장합니다. 깨어나면 항상 조건을 확인하기 때문입니다.

7.4 모니터의 장점과 한계

모니터는 세마포 대비 큰 장점을 제공했습니다. 캡슐화로 공유 데이터가 보호됩니다. 외부에서 직접 접근할 수 없습니다. 상호 배제가 자동으로 보장됩니다. 프로그래머가 락을 명시적으로 관리할 필요가 없습니다. 조건이 명확히 표현됩니다. not_full, not_empty 같은 이름이 의도를 드러냅니다. 구조화된 설계로 오류 가능성이 줄어듭니다. 세마포의 P와 V 순서를 잘못 쓸 염려가 없습니다. 모든 것이 모니터 내부에 캡슐화되어 있습니다.

그러나 모니터도 한계가 있었습니다. 언어 차원의 지원이 필요합니다. Concurrent Pascal, Mesa, Modula 같은 언어들이 모니터를 직접 지원했습니다. 그러나 C 같은 언어에서는 라이브러리로 구현해야 했습니다. 자동 락 획득이 불가능했습니다. 프로그래머가 명시적으로 관리해야 했습니다.

여러 조건을 동시에 기다리기 어렵습니다. 각 조건 변수는 하나의 조건만 나타냅니다. 복잡한 조건은 표현하기 어렵습니다. 모니터 함수 내에서 다른 모니터 함수를 호출하면 교착상태가 발생할 수 있습니다. 같은 모니터의 락을 이미 잡고 있는 상태에서 다시 획득하려 하기 때문입니다.